Pointers in C

What is a Pointer in C?

C pointer is the derived data type that is used to store the address of another variable and can also be used to access and manipulate the variable's data stored at that location. The pointers are considered as derived data types.

With pointers, you can access and modify the data located in the memory, pass the data efficiently between the functions, and create dynamic data structures like linked lists, trees, and graphs.

Pointer Declaration

To declare a pointer, use the **dereferencing operator (*)** followed by the data type.

Syntax

The general form of a pointer variable declaration is –

```
type *var-name;
```

Here, **type** is the pointer's base type; it must be a valid C data type and **var-name** is the name of the pointer variable. The asterisk * used to declare a pointer is the same asterisk used for multiplication. However, in this statement the asterisk is being used to designate a variable as a pointer.

Example of Valid Pointer Variable Declarations

Take a look at some of the valid pointer declarations –

```
int    *ip;    /* pointer to an integer */
double *dp;    /* pointer to a double */
float  *fp;    /* pointer to a float */
char   *ch     /* pointer to a character */
```

The actual data type of the value of all pointers, whether integer, float, character, or otherwise, is the same, a long hexadecimal number that represents a memory address. The only difference between pointers of different data types is the data type of the variable or constant that the pointer points to.

Pointer Initialization

After declaring a pointer variable, you need to initialize it with the address of another variable using the **address of (&) operator**. This process is known as **referencing a pointer**.

Syntax

The following is the syntax to initialize a pointer variable –

```
pointer_variable = &variable;
```

Example

Here is an example of pointer initialization –

```
int x = 10;
int *ptr = &x;
```

Here, **x** is an integer variable, **ptr** is an integer pointer. The pointer **ptr** is being initialized with **x**.

Referencing and Dereferencing Pointers

A pointer references a location in memory. Obtaining the value stored at that location is known as **dereferencing the pointer**.

In C, it is important to understand the purpose of the following two operators in the context of pointer mechanism –

- **The & Operator** – It is also known as the "Address-of operator". It is used for Referencing which means taking the address of an existing variable (using **&**) to set a pointer variable.
- **The * Operator** – It is also known as the "dereference operator". **Dereferencing** a pointer is carried out using the *** operator** to get the value from the memory address that is pointed by the pointer.

Pointers are used to pass parameters by reference. This is useful if a programmer wants a function's modifications to a parameter to be visible to the function's caller. This is also

useful for returning multiple values from a function.

Access and Manipulate Values using Pointer

The value of the variable which is pointed by a pointer can be accessed and manipulated by using the pointer variable. You need to use the asterisk (*) sign with the pointer variable to access and manipulate the variable's value.

Example

In the below example, we are taking an integer variable with its initial value and changing it with the new value.

</> Open Compiler

```
#include <stdio.h>

int main() {
  int x = 10;

  // Pointer declaration and initialization
  int * ptr = & x;

  // Printing the current value
  printf("Value of x = %d\n", * ptr);

  // Changing the value
  * ptr = 20;

  // Printing the updated value
  printf("Value of x = %d\n", * ptr);

  return 0;
}
```

Output

```
Value of x = 10
Value of x = 20
```

How to Use Pointers?

To use the pointers in C language, you need to declare a pointer variable, then initialize it with the address of another variable, and then you can use it by dereferencing to get and change the value of the variables pointed by the pointer.

You can use pointers with any type of variable such as integer, float, string, etc. You can also use pointers with derived data types such as array, structure, union, etc.

Example

In the below example, we are using pointers for getting values of different types of variables.

</> Open Compiler

```
#include <stdio.h>

int main() {
  int x = 10;
  float y = 1.3f;
  char z = 'p';

  // Pointer declaration and initialization
  int * ptr_x = & x;
  float * ptr_y = & y;
  char * ptr_z = & z;

  // Printing the values
  printf("Value of x = %d\n", * ptr_x);
  printf("Value of y = %f\n", * ptr_y);
  printf("Value of z = %c\n", * ptr_z);

  return 0;
}
```

Output

```
Value of x = 10
Value of y = 1.300000
```

```
Value of z = p
```

Size of a Pointer Variable

The memory (or, size) occupied by a pointer variable does not depend on the type of the variable it is pointing to. The size of a pointer depends on the system architecture.

Example

In the below example, we are printing the size of different types of pointers:

</> Open Compiler

```
#include <stdio.h>

int main() {
  int x = 10;
  float y = 1.3f;
  char z = 'p';

  // Pointer declaration and initialization
  int * ptr_x = & x;
  float * ptr_y = & y;
  char * ptr_z = & z;

  // Printing the size of pointer variables
  printf("Size of integer pointer : %lu\n", sizeof(ptr_x));
  printf("Size of float pointer : %lu\n", sizeof(ptr_y));
  printf("Size of char pointer : %lu\n", sizeof(ptr_z));

  return 0;
}
```

Output

```
Size of integer pointer : 8
Size of float pointer : 8
Size of char pointer : 8
```

Examples of C Pointers

Practice the following examples to learn the concept of pointers –

Example 1: Using Pointers in C

The following example shows how you can use the **&** and ***** operators to carry out pointer-related opeartions in C –

</> Open Compiler

```
#include <stdio.h>

int main(){

   int  var = 20;   /* actual variable declaration */
   int  *ip;        /* pointer variable declaration */

   ip = &var;   /* store address of var in pointer variable*/

   printf("Address of var variable: %p\n", &var);

   /* address stored in pointer variable */
   printf("Address stored in ip variable: %p\n", ip);

   /* access the value using the pointer */
   printf("Value of *ip variable: %d\n", *ip );

   return 0;
}
```

Output

Execute the code and check its output –

```
Address of var variable: 0x7ffea76fc63c
Address stored in ip variable: 0x7ffea76fc63c
Value of *ip variable: 20
```

Example: Print Value and Address of an Integer

We will declare an int variable and display its value and address –

```
#include <stdio.h>

int main(){

   int var = 100;

   printf("Variable: %d \t Address: %p", var, &var);

   return 0;
}
```

Output

Run the code and check its output –

```
Variable: 100   Address: 0x7ffc62a7b844
```

Example: Integer Pointer

In this example, the address of **var** is stored in the **intptr** variable with & operator

```
#include <stdio.h>

int main(){

   int var = 100;
   int *intptr = &var;

   printf("Variable: %d \nAddress of Variable: %p \n\n", var, &var);
   printf("intptr: %p \nAddress of intptr: %p \n\n", intptr, &intptr);

   return 0;
}
```

Output

Run the code and check its output –

```
Variable: 100
Address of Variable: 0x7ffdcc25860c

intptr: 0x7ffdcc25860c
Address of intptr: 0x7ffdcc258610
```

Example 5

Now let's take an example of a float variable and find its address –

</> Open Compiler

```
#include <stdio.h>

int main(){

   float var1 = 10.55;

   printf("var1: %f \n", var1);
   printf("Address of var1: %d", &var1);
}
```

Output

Run the code and check its output –

```
var1: 10.550000
Address of var1: 1512452612
```

We can see that the address of this variable (any type of variable for that matter) is an integer. So, if we try to store it in a pointer variable of "float" type, see what happens –

```
float var1 = 10.55;
int *intptr = &var1;
```

The compiler doesn't accept this, and reports the following **error** –

```
initialization of 'int *' from incompatible pointer type
'float *' [-Wincompatible-pointer-types]
```

Note: The type of a variable and the type of its pointer must be same.

In C, variables have specific data types that define their size and how they store values. Declaring a pointer with a matching type (e.g., float *) enforces "type compatibility" between the pointer and the data it points to.

Different data types occupy different amounts of memory space in C. For example, an "int" typically takes 4 bytes, while a "float" might take 4 or 8 bytes depending on the system. Adding or subtracting integers from pointers moves them in memory based on the size of the data they point to.

Example: Float Pointer

In this example, we declare a variable "floatptr" of "float *" type.

</> Open Compiler

```
#include <stdio.h>

int main(){

   float var1 = 10.55;
   float *floatptr = &var1;

   printf("var1: %f \nAddress of var1: %p \n\n",var1, &var1);
   printf("floatptr: %p \nAddress of floatptr: %p \n\n", floatptr, &floatptr);
   printf("var1: %f \nValue at floatptr: %f", var1, *floatptr);

   return 0;
}
```

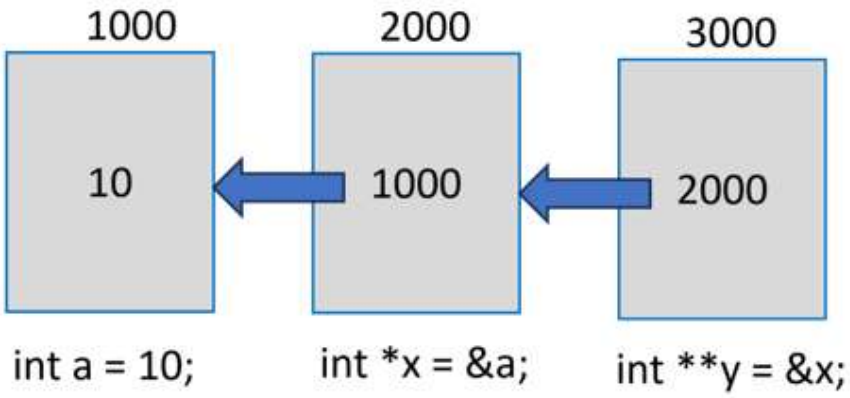
Output

```
var1: 10.550000
Address of var1: 0x7ffc6daeb46c
```

```
floatptr: 0x7ffc6daeb46c
Address of floatptr: 0x7ffc6daeb470
```

Pointer to Pointer

We may have a pointer variable that stores the address of another pointer itself.



In the above figure, "a" is a normal "int" variable, whose pointer is "x". In turn, the variable stores the address of "x".

Note that "y" is declared as "int **" to indicate that it is a pointer to another pointer variable. Obviously, "y" will return the address of "x" and "*y" is the value in "x" (which is the address of "a").

To obtain the value of "a" from "y", we need to use the expression "**y". Usually, "y" will be called as the **pointer to a pointer**.

Example

Take a look at the following example –

</> Open Compiler

```
#include <stdio.h>

int main(){

   int var = 10;
   int *intptr = &var;
   int **ptrptr = &intptr;

   printf("var: %d \nAddress of var: %d \n\n",var, &var);
```

```
   printf("inttptr: %d \nAddress of inttptr: %d \n\n", intptr, &intptr);
   printf("var: %d \nValue at intptr: %d \n\n", var, *intptr);
   printf("ptrptr: %d \nAddress of ptrtptr: %d \n\n", ptrptr, &ptrptr);
   printf("intptr: %d \nValue at ptrptr: %d \n\n", intptr, *ptrptr);
   printf("var: %d \n*intptr: %d \n**ptrptr: %d", var, *intptr, **ptrptr);

   return 0;
}
```

Output

Run the code and check its output –

```
var: 10
Address of var: 951734452

inttptr: 951734452
Address of inttptr: 951734456

var: 10
Value at intptr: 10

ptrptr: 951734456
Address of ptrtptr: 951734464
intptr: 951734452
Value at ptrptr: 951734452

var: 10
*intptr: 10
**ptrptr: 10
```

You can have a **pointer to an array** as well as a derived type defined with **struct.** Pointers have important applications. They are used while calling a function by passing the reference. Pointers also help in overcoming the limitation of a function's ability to return only a single value. With pointers, you can get the effect of returning multiple values or arrays.

NULL Pointers

It is always a good practice to assign a NULL value to a pointer variable in case you do not have an exact address to be assigned. This is done at the time of variable declaration. A pointer that is assigned NULL is called a **null** pointer.

The NULL pointer is a constant with a value of "0" defined in several standard libraries.

Example

Consider the following program –

```
</>                                                    Open Compiler

#include <stdio.h>

int main(){

   int *ptr = NULL;

   printf("The value of ptr is : %x\n", ptr);

   return 0;
}
```

Output

When the above code is compiled and executed, it produces the following result –

```
The value of ptr is 0
```

In most operating systems, programs are not permitted to access memory at address "0" because that memory is reserved by the operating system.

The memory address "0" has a special significance; it signals that the pointer is not intended to point to an accessible memory location. But by convention, if a pointer contains the null (zero) value, it is assumed to point to nothing.

To check for a null pointer, you can use an **if** statement as follows –

```
if(ptr)     /* succeeds if p is not null */
if(!ptr)    /* succeeds if p is null */
```

Address of the Variables

As you know, every variable is a memory location and every memory location has its address defined which can be accessed using the ampersand (&) operator, which denotes

an address in memory.

Example

Consider the following example, which prints the address of the variables defined –

```
#include <stdio.h>

int main(){

   int  var1;
   char var2[10];

   printf("Address of var1 variable: %x\n", &var1);
   printf("Address of var2 variable: %x\n", &var2);

   return 0;
}
```

Output

When the above code is compiled and executed, it will print the address of the variables –

```
Address of var1 variable: 61e11508
Address of var2 variable: 61e1150e
```

Pointers in Detail

Pointers have many but easy concepts and they are very important to C programming. The following important pointer concepts should be clear to any C programmer –

Sr.No	Concept & Description
1	**Pointer arithmetic** There are four arithmetic operators that can be used in pointers: ++, --, +, -
2	**Array of pointers** You can define arrays to hold a number of pointers.

3	**Pointer to pointer** C allows you to have pointer on a pointer and so on.
4	**Passing pointers to functions in C** Passing an argument by reference or by address enable the passed argument to be changed in the calling function by the called function.
5	**Return pointer from functions in C** C allows a function to return a pointer to the local variable, static variable, and dynamically allocated memory as well.

Applications of Pointers in C

One of the most important features of C is that it provides low-level memory access with the concept of pointers. A **pointer** is a variable that stores the address of another variable in the memory.

The provision of pointers has many applications such as passing arrays and struct type to a function and dynamic memory allocation, etc. In this chapter, we will explain some important applications of pointers in C.

To Access Array Elements

Array elements can also be accessed through the pointer. You need to declare and initialize a pointer to an array and using it you can access each element by incrementing the pointer variable by 1.

The pointer to an array is the address of its 0th element. When the array pointer is incremented by 1, it points to the next element in the array.

Example

The following example demonstrates how you can traverse an array with the help of its pointer.

</> Open Compiler

```
#include <stdio.h>

int main(){

   int arr[] = {1,2,3,4,5};
   int *ptr = arr;

   for(int i = 0; i <= 4; i++){
      printf("arr[%d]: %d\n", i, *ptr);
      ptr++;
   }

   return 0;
}
```

Output

Run the code and check its output –

```
arr[0]: 1
arr[1]: 2
arr[2]: 3
arr[3]: 4
arr[4]: 5
```

For Allocating Memory Dynamically

One of the most important applications of C pointers is to declare memory for the variables dynamically. There are various situations, where static memory allocation cannot solve the problem, such as dealing with large size of arrays, structures having n numbers of students and employees, etc.

Thus, whenever you need to allocate memory dynamically, pointers play an important role in it. C language provides some of the functions to allocate and release the memory dynamically. The functions are:

- **`malloc()` function**
  Allocates an array of num elements each of which size in bytes will be size.
- **`calloc()` function**
  Allocates an array of num bytes and leaves them uninitialized.
- **`realloc()` function**
  Reallocates memory extending it up to newsize.

1. The malloc() Function

This function is defined in the "stdlib.h" header file. It allocates a block memory of the required size and returns a **void** pointer.

```
void *malloc (size)
```

The size parameter refers to the block of memory in bytes. To allocate the memory required for a specified data type, you need to use the typecasting operator. For example, the following snippet allocates the memory required to store an int type.

```
int *ptr;
```

```
ptr = (int * ) malloc (sizeof (int));
```

Here we need to define a pointer to character without defining how much memory is required and later, based on requirement, we can allocate memory.

Example

In this example, we use the malloc() function to allocate the required memory to store a string (instead of declaring a **char** array of a fixed size) −

</> Open Compiler

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(){

   char *name;
   name = (char *) malloc(strlen("TutorialsPoint"));

   strcpy(name, "TutorialsPoint");

   if(name == NULL) {
      fprintf(stderr, "Error - unable to allocate required memory\n");
   } else {
      printf("Name = %s\n", name );
   }
}
```

Output

When the above code is compiled and executed, it produces the following output −

```
Name = TutorialsPoint
```

2. The calloc() Function

The C library function **"calloc"** (stands for contiguous allocation) allocates the requested memory and returns a pointer to it.

```
void *calloc(n, size);
```

Where **"n"** is the number of elements to be allocated and **"size"** is the byte size of each element.

The following snippet allocates the memory required to store 10 integer types.

```
int *ptr;
ptr = (int *) calloc(25, sizeof(int));
```

3. The realloc() Function

The **realloc()** function in C is used to dynamically change the memory allocation of a previously allocated memory. You can increase or decrease the size of an allocated memory block by calling the **realloc()** function.

```
void *realloc(*ptr, size);
```

The first parameter **"ptr"** is the pointer to a memory block previously allocated with **malloc, calloc** or **realloc** to be reallocated.

Dynamic memory allocation technique is extensively used in complex linear and non−linear data structures such as linked lists and trees, which are employed in operating system software.

For Passing Arguments as Reference

When a function is called by reference, the address of the actual argument variables passed, instead of their values.

Passing a pointer to a function has two **advantages** −

First, it overcomes the limitation of pass by value. Changes to the value inside the called function are done directly at the address stored in the pointer. Hence, we can manipulate the variables in one scope from another.

Second, it also overcomes the limitation of a function in that it can return only one expression. By passing pointers, the effect of processing a function takes place directly at the address. Secondly, more than one value can be returned if we return the pointer of an array or struct variable.

Example

The following function receives the reference of two variables whose values are to be swapped.

```
/* function definition to swap the values */

int swap(int *x, int *y){
   int z;
   z = *x;    /* save the value at address x */
   *x = *y;   /* put y into x */
   *y = z;    /* put z into y */

   return 0;
}
```

Example

The main() function has two variables "a" and "b", their addresses are passed as arguments to the swap() function.

```
#include <stdio.h>

int swap(int *x, int *y);

int main(){

   /* local variable definition */
   int a = 10;
   int b = 20;

   printf("Before swap, value of a : %d\n", a);
   printf("Before swap, value of b : %d\n", b);

   /* calling a function to swap the values */
   swap(&a, &b);

   printf("After swap, value of a: %d\n", a);
   printf("After swap, value of b: %d\n", b);
```

```
   return 0;
}
```

Output

When executed, it will produce the following output –

```
Before swap, value of a: 10
Before swap, value of b: 20
After swap, value of a: 20
After swap, value of b: 10
```

In this program, we have been able to swap the values of two variables out of the scope of a function to which they have been passed, and we could overcome the limitation of the function's ability to pas only one expression.

For Passing an Array to Function

Let us use these characteristics for passing the array by reference. In the main() function, we declare an array and pass its address to the **max()** function.

The **max()** function traverses the array using the pointer and returns the largest number in the array, back to the main() function.

Example

Take a look at the following example –

</> Open Compiler

```
#include <stdio.h>

int max(int *arr, int length);

int main(){

   int arr[] = {10, 34, 21, 78, 5};
   int length = sizeof(arr)/sizeof(int);

   int maxnum = max(arr, length);
      printf("max: %d", maxnum);
}
```

```
int max(int *arr, int length){

   int max = *arr;

   for (int i = 0; i < length; i++){
      printf("arr[%d]: %d\n", i, (*arr));

      if ((*arr)>max)
         max = (*arr);
      arr++;
   }
   return max;
}
```

Output

Run the code and check its output –

```
arr[0]: 10
arr[1]: 34
arr[2]: 21
arr[3]: 78
arr[4]: 5
max: 78
```

The max() function receives the address of the array from the main() function in the pointer **"arr".** Each time, when it is incremented, it points to the next element in the original array.

For Returning Multiple Values from a Function

In C language, the functions can have only one return statement to return one value at a time. With the help of C pointers, you can return multiple values from a function by passing arguments as references.

Example

The following example demonstrates how you can return multiple values with the help of C pointers.

</> Open Compiler

```c
#include <stdio.h>

// Creating a function to find
// addition and subtraction
// of two numbers
void funAddSub(int a, int b, int* add, int* sub) {
  *add = a + b;
  *sub = a - b;
}

int main() {
  int num1 = 10;
  int num2 = 3;

  // Variables to store results
  int res1, res2;

  // Calling function to get add and sub
  // by passing the address of res1 and res2
  funAddSub(num1, num2, &res1, &res2);

  // Printing the result
  printf("Addition is %d and subtraction is %d", res1, res2);

  return 0;
}
```

Output

```
Addition is 13 and subtraction is 7
```

Pointers and Arrays in C

In C programming, the concepts of arrays and pointers have a very important role. There is also a close association between the two. In this chapter, we will explain in detail the relationship between arrays and pointers in C programming.

Arrays in C

An array in C is a homogenous collection of elements of a single data type stored in a continuous block of memory. The size of an array is an integer inside square brackets, put in front of the name of the array.

Declaring an Array

To declare an array, the following **syntax** is used –

```
data_type arr_name[size];
```

Each element in the array is identified by a unique incrementing index, starting from "0". An array can be declared and initialized in different ways.

You can declare an array and then initialize it later in the code, as and when required. **For example –**

```
int arr[5];
...
...
a[0] = 1;
a[1] = 2;
...
...
```

You can also declare and initialize an array at the same time. The values to be stored are put as a comma separated list inside curly brackets.

```
int a[5] = {1, 2, 3, 4, 5};
```

Note: When an array is initialized at the time of declaration, mentioning its size is optional, as the compiler automatically computes the size. Hence, the following statement is also valid –

```
int a[] = {1, 2, 3, 4, 5};
```

Example: Lower Bound and Upper Bound of an Array

All the elements in an array have a positional index, starting from "0". The **lower bound** of the array is always "0", whereas the upper bound is **"size − 1"**. We can use this property to traverse, assign, or read inputs into the array subscripts with a loop.

Take a look at this following example −

</> Open Compiler

```
#include <stdio.h>

int main(){

   int a[5], i;

   for(i = 0; i <= 4; i++){
      scanf("%d", &a[i]);
   }

   for(i = 0; i <= 4; i++){
      printf("a[%d] = %d\n",i,  a[i]);
   }

   return 0;
}
```

Output

Run the code and check its output −

```
a[0] = 712952649
a[1] = 32765
a[2] = 100
a[3] = 0
a[4] = 4096
```

Pointers in C

A pointer is a variable that stores the address of another variable. In C, the symbol **(&)** is used as the **address-of operator**. The value returned by this operator is assigned to a pointer.

To declare a variable as a pointer, you need to put an asterisk (*) before the name. Also, the type of pointer variable must be the same as the type of the variable whose address it stores.

In this code snippet, **"b"** is an integer pointer that stores the address of an integer variable **"a"** −

```
int a = 5;
int *b = &a;
```

In case of an array, you can assign the address of its 0^{th} element to the pointer.

```
int arr[] = {1, 2, 3, 4, 5};
int *b = &arr[0];
```

In C, the name of the array itself resolves to the address of its 0^{th} element. It means, in the above case, we can use **"arr"** as equivalent to **"&[0]":**

```
int *b = arr;
```

Example: Increment Operator with Pointer

Unlike a normal numeric variable (where the increment operator "++" increments its value by 1), the increment operator used with a pointer increases its value by the **sizeof** its data type.

Hence, an **int** pointer, when incremented, increases by 4.

</> Open Compiler

```
#include <stdio.h>

int main(){
```

```
   int a = 5;
   int *b = &a;

   printf("Address of a: %d\n", b);
   b++;
   printf("After increment, Address of a: %d\n", b);

   return 0;
}
```

Output

Run the code and check its output –

```
Address of a: 6422036
After increment, Address of a: 6422040
```

The Dereference Operator in C

In C, the "*" symbol is used as the **dereference operator.** It returns the value stored at the address to which the pointer points.

Hence, the following statement returns "5", which is the value stored in the variable **"a",** the variable that **"b"** points to.

```
int a = 5;
int *b = &a;
printf("value of a: %d\n", *b);
```

Note: In case of a char pointer, it will increment by 1; in case of a **double** pointer, it will increment by 8; and in case of a **struct** type, it increments by the **sizeof value** of that struct type.

Example: Traversing an Array Using a Pointer

We can use this property of the pointer to traverse the array element with the help of a pointer.

</> Open Compiler

```c
#include <stdio.h>

int main(){

   int arr[5] = {1, 2, 3, 4, 5};
   int *b = arr;

   printf("Address of a[0]: %d value at a[0] : %d\n",b, *b);

   b++;
   printf("Address of a[1]: %d value at a[1] : %d\n", b, *b);

   b++;
   printf("Address of a[2]: %d value at a[2] : %d\n", b, *b);

   b++;
   printf("Address of a[3]: %d value at a[3] : %d\n", b, *b);

   b++;
   printf("Address of a[4]: %d value at a[4] : %d\n", b, *b);

   return 0;
}
```

Output

Run the code and check its output –

```
address of a[0]: 6422016 value at a[0] : 1
address of a[1]: 6422020 value at a[1] : 2
address of a[2]: 6422024 value at a[2] : 3
address of a[3]: 6422028 value at a[3] : 4
address of a[4]: 6422032 value at a[4] : 5
```

Points to Note

It may be noted that –

- **"&arr[0]"** is equivalent to **"b"** and **"arr[0]"** to **"*b".**

- Similarly, **"&arr[1]"** is equivalent to **"b + 1"** and **"arr[1]"** is equivalent to **"*(b + 1)".**
- Also, **"&arr[2]"** is equivalent to **"b + 2"** and **"arr[2]"** is equivalent to **"*(b+2)".**
- In general, **"&arr[i]"** is equivalent to **"b + I"** and **"arr[i]"** is equivalent to **"*(b+i)".**

Example: Traversing an Array using the Dereference Operator

We can use this property and use a loop to traverse the array with the **dereference operator.**

</> Open Compiler

```
#include <stdio.h>

int main(){

   int arr[5] = {1, 2, 3, 4, 5};
   int *b = arr;
   int i;

   for(i = 0; i <= 4; i++){
      printf("a[%d] = %d\n",i,  *(b+i));
   }

   return 0;
}
```

Output

Run the code and check its output –

```
a[0] = 1
a[1] = 2
a[2] = 3
a[3] = 4
a[4] = 5
```

You can also increment the pointer in every iteration and obtain the same result –

```
for(i = 0; i <= 4; i++){
   printf("a[%d] = %d\n", i, *b);
   b++;
}
```

The concept of arrays and pointers in C has a close relationship. You can use pointers to enhance the efficiency of a program, as pointers deal directly with the memory addresses. Pointers can also be used to handle multi−dimensional arrays.

Pointer Arithmetics in C

A pointer variable in C stores the address of another variable. The address is always an integer. So, can we perform arithmetic operations such as addition and subtraction on the pointers? In this chapter, we will explain which arithmetic operators use pointers in C as operands, and which operations are not defined to be performed with pointers.

C pointers arithmetic operations are different from the general arithmetic operations. The following are some of the important pointer arithmetic operations in C:

- Increment and Decrement of a Pointer
- Addition and Subtraction of Integer to Pointer
- Subtraction of Pointers
- Comparison of Pointers

Let us discuss all these pointer arithmetic operations in detail with the help of examples.

Increment and Decrement of a Pointer

We know that "++" and "--" are used as the increment and decrement operators in C. They are unary operators, used in prefix or postfix manner with numeric variable operands, and they increment or decrement the value of the variable by one.

Assume that an integer variable "x" is created at address 1000 in the memory, with 10 as its value. Then, "x++" makes the value of "x" as 11.

```
int x = 10;   // created at address 1000

x++;          // x becomes 11
```

What happens if we declare "y" as pointer to "x" and increment "y" by 1 (with "y++")? Assume that the address of "y" itself is 2000.

```
int x = 10;   // created at address 1000

// "y" is created at address 2000
// it holds 1000 (address of "x")
int *y = &x ;

y++;          // y becomes 1004
```

Since the variable "y" stores 1000 (the address of "x"), we expect it to become 1001 because of the "++" operator, but it increments by 4, which is the size of "int" variable.

The is why because, if the address of "x" is 1000, then it occupies 4 bytes: 1000, 1001, 1002 and 1003. Hence, the next integer can be put only in 1004 and not before it. Hence "y" (the pointer to "x") becomes 1004 when incremented.

Example of Incrementing a Pointer

The following example shows how you can increment a pointer −

</> Open Compiler

```
#include <stdio.h>

int main(){

   int x = 10;
   int *y = &x;

   printf("Value of y before increment: %d\n", y);

   y++;

   printf("Value of y after increment: %d", y);
}
```

Output

Run the code and check its output −

```
Value of y before increment: 6422036
Value of y after increment: 6422040
```

You can see that the value has increased by 4. Similarly, the "--" operator decrements the value by the size of the data type.

Example of Decrementing a Pointer

Let us change the types of "x" and "y" to "double" and "float" and see the effect of decrement operator.

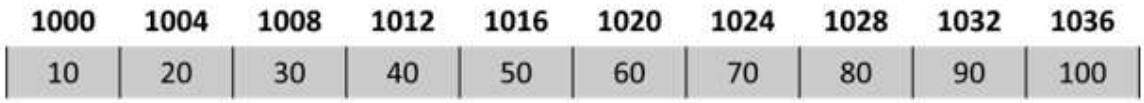
</> Open Compiler

```
#include <stdio.h>

int main(){

   double x = 10;
   double *y = &x;

   printf("value of y before decrement: %ld\n", y);

   y--;

   printf("value of y after decrement: %ld", y);
}
```

Output

```
Value of y before decrement: 6422032
Value of y after decrement: 6422024
```

When an array is declared, the elements are stored in adjacent memory locations. In case of "int" array, each array subscript is placed apart by 4 bytes, as the following figure shows –



Hence, if a variable stores the address of 0th element of the array, then the "increment" takes it to the 1st element.

Example of Traversing an Array by Incrementing Pointer

The following example shows how you can traverse an array by incrementing a pointer successively –

</> Open Compiler

```c
#include <stdio.h>

int main(){

   int a[]= {10, 20, 30, 40, 50, 60, 70, 80, 90, 100};
   int len = sizeof(a)/sizeof(int);
   int *x = a;
   int i = 0;

   for(i = 0; i < len; i++){
      printf("Address of subscript %d = %d Value = %d\n", i, x, *x);
      x++;
   }

   return 0;
}
```

Output

Run the code and check its output –

```
Address of subscript 0 = 6421984 Value = 10
Address of subscript 1 = 6421988 Value = 20
Address of subscript 2 = 6421992 Value = 30
Address of subscript 3 = 6421996 Value = 40
Address of subscript 4 = 6422000 Value = 50
Address of subscript 5 = 6422004 Value = 60
Address of subscript 6 = 6422008 Value = 70
Address of subscript 7 = 6422012 Value = 80
Address of subscript 8 = 6422016 Value = 90
Address of subscript 9 = 6422020 Value = 100
```

Addition and Subtraction of Integer to Pointer

An integer value can be added and subtracted to a pointer. When an integer is added to a pointer, the pointer points to the next memory address. Similarly, when an integer is subtracted from a pointer, the pointer points to the previous memory location.

Addition and subtraction of an integer to a pointer does not add and subtract that value to the pointer, multiplication with the size of the data type is added or subtracted to the pointer.

For example, there is an integer pointer variable ptr and it is pointing to an address 123400, if you add 1 to the ptr (ptr+1), it will point to the address 123404 (size of an integer is 4).

Let's evaluate it,

```
ptr = 123400
ptr = ptr + 1
ptr = ptr + sizeof(int)*1
ptr = 123400 + 4
ptr = 123404
```

Example of Adding Value to a Pointer

In the following example, we are declaring an array and pointer to an array. Initializing the pointer with the first element of the array and then adding an integer value (2) to the pointer to get the third element of the array.

</> Open Compiler

```c
#include <stdio.h>

int main() {
  int int_arr[] = {12, 23, 45, 67, 89};
  int *ptrArr = int_arr;

  printf("Value at ptrArr: %d\n", *ptrArr);

  // Adding 2 in ptrArr
  ptrArr = ptrArr + 2;

  printf("Value at ptrArr after adding 2: %d\n", *ptrArr);

  return 0;
}
```

Output

```
Value at ptrArr: 12
```

```
Value at ptrArr after adding 2: 45
```

Example of Subtracting Value to a Pointer

In the following example, we are declaring an array and pointer to an array. Initializing the pointer with the last element of the array and then subtracting an integer value (2) from the pointer to get the third element of the array.

</> Open Compiler

```c
#include <stdio.h>

int main() {
  int int_arr[] = {12, 23, 45, 67, 89};
  int *ptrArr = &int_arr[4]; // points to last element

  printf("Value at ptrArr: %d\n", *ptrArr);

  // Subtracting 2 in ptrArr
  ptrArr = ptrArr - 2;

  printf("Value at ptrArr after adding 2: %d\n", *ptrArr);

  return 0;
}
```

Output

```
Value at ptrArr: 89
Value at ptrArr after adding 2: 45
```


Subtraction of Pointers

We are familiar with the "**+**" and "**−**" operators when they are used with regular numeric operands. However, when you use these operators with pointers, they behave in a little different way.

Since pointers are fairly large integers (especially in modern 64-bit systems), addition of two pointers is meaningless. When we add a 1 to a pointer, it points to the next location where an integer may be stored. Obviously, when we add a pointer (itself a large integer), the location it points may not be in the memory layout.

However, subtraction of two pointers is realistic. It returns the number of data types that can fit in the two pointers.

Example of Subtracting Two Pointers

Let us take the array in the previous example and perform the subtraction of pointers of a[0] and a[9]

</> Open Compiler

```
#include <stdio.h>

int main(){

   int a[]= {10, 20, 30, 40, 50, 60, 70, 80, 90, 100};
   int *x = &a[0]; // zeroth element
   int *y = &a[9]; // last element

   printf("Add of a[0]: %ld add of a[9]: %ld\n", x, y);
   printf("Subtraction of two pointers: %ld", y-x);

}
```

Output

Run the code and check its output –

```
Add of a[0]: 140729162482768 add of a[9]: 140729162482804
Subtraction of two pointers: 9
```

It can be seen that the numerical difference between the two integers is 36; it suggests that the subtraction is 9, because it can accommodate 9 integers between the two pointers.

Comparison of Pointers

Pointers may be compared by using relational operators such as "==", "<", and ">". If "p1" and "p2" point to variables that are related to each other (such as elements of the same array), then "p1" and "p2" can be meaningfully compared.

Example of Comparing Pointers

In the following example, we are declaring two pointers and initializing them with the first and last elements of the array respectively. We will keep incrementing the first variable pointer as long as the address to which it points is either less than or equal to the address of the last element of the array, which is "&var[MAX − 1]" (i.e., the second pointer).

</> Open Compiler

```
#include <stdio.h>

const int MAX = 3;

int main() {
  int var[] = {10, 100, 200};
  int i, *ptr1, *ptr2;

  // Initializing pointers
  ptr1 = var;
  ptr2 = &var[MAX - 1];

  while (ptr1 <= ptr2) {
    printf("Address of var[%d] = %p\n", i, ptr1);
    printf("Value of var[%d] = %d\n", i, *ptr1);

    /* point to the previous location */
    ptr1++;
    i++;
  }

  return 0;
}
```

Output

Run the code and check its output −

```
Address of var[0] = 0x7ffe7101498c
Value of var[0] = 10
Address of var[1] = 0x7ffe71014990
Value of var[1] = 100
Address of var[2] = 0x7ffe71014994
Value of var[2] = 200
```

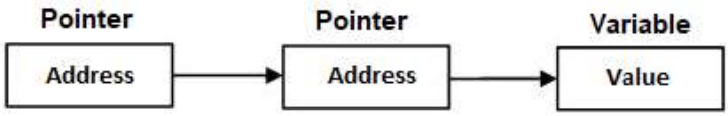
Pointer to Pointer (Double Pointer) in C

What is a Double Pointer in C?

A **pointer to pointer** which is also known as a **double pointer** in C is used to store the address of another pointer.

A variable in C that stores the address of another variable is known as a **pointer.** A pointer variable can store the address of any type including the primary data types, arrays, struct types, etc. Likewise, a pointer can store the address of another pointer too, in which case it is called **"pointer to pointer"** (also called **"double pointer"**).

A "pointer to a pointer" is a form of **multiple indirection** or a **chain of pointers.** Normally, a pointer contains the address of a variable. When we define a "pointer to a pointer", the first pointer contains the address of the second pointer, which points to the location that contains the actual value as shown below –



Declaration of Pointer to a Pointer

The declaration of a **pointer to pointer** (**double pointer**) is similar to the declaration of a pointer, the only difference is that you need to use an **additional asterisk** (*) before the pointer variable name.

Example

For example, the following declaration declares a "pointer to a pointer" of type **int –**

```
int **var;
```

When a target value is indirectly pointed to by a "pointer to a pointer", accessing that value requires that the asterisk operator be applied twice.

Explore our **latest online courses** and learn new skills at your own pace. Enroll and become a certified expert to boost your career.

Example of Pointer to Pointer (Double Pointer)

The following example demonstrates the declaration, initialization, and using pointer to pointer (double pointer) in C:

</>

Open Compiler

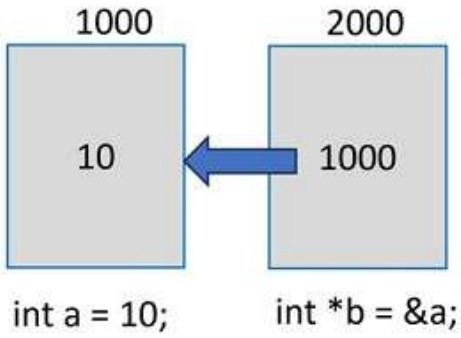
```c
#include <stdio.h>

int main() {
  // An integer variable
  int a = 100;

  // Pointer to integer
  int *ptr = &a;

  // Pointer to pointer (double pointer)
  int **dptr = &ptr;

  printf("Value of 'a' is : %d\n", a);
  printf("Value of 'a' using pointer (ptr) is : %d\n", *ptr);
  printf("Value of 'a' using double pointer (dptr) is : %d\n", **dptr);

  return 0;
}
```

Output

```
Value of 'a' is : 100
Value of 'a' using pointer (ptr) is : 100
Value of 'a' using double pointer (dptr) is : 100
```

How Does a Normal Pointer Work in C?

Assume that an integer variable **"a"** is located at an arbitrary address 1000. Its pointer variable is **"b"** and the compiler allocates it the address 2000. The following image presents a visual depiction –



Let us declare a pointer to **int** type and store the address of an **int** variable in it.

```
int a = 10;
int *b = &a;
```

The **dereference operator** fetches the value via the pointer.

```
printf("a: %d \nPointer to 'a' is 'b': %d \nValue at 'b': %d", a, b, *b);
```

Example

Here is the complete program that shows how a normal pointer works –

</> Open Compiler

```
#include <stdio.h>

int main(){

   int a = 10;
   int *b = &a;
   printf("a: %d \nPointer to 'a' is 'b': %d \nValue at 'b': %d", a, b, *b);

   return 0;
}
```

Output

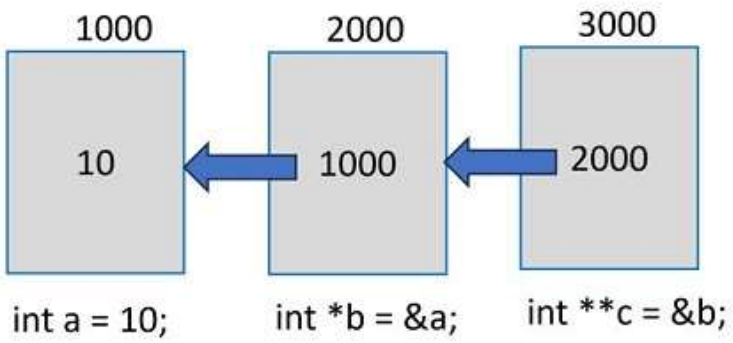
It will print the value of **int** variable, its address, and the value obtained by the dereference pointer –

```
a: 10
Pointer to 'a' is 'b': 6422036
Value at 'b': 10
```

How Does a Double Pointer Work?

Let us now declare a pointer that can store the address of "b", which itself is a pointer to **int** type written as **"int *".**

Let's assume that the compiler also allocates it the address 3000.



Hence, **"c"** is a pointer to a pointer to **int,** and should be declared as **"int **".**

```
int **c = &b;
printf("b: %d \nPointer to 'b' is 'c': %d \nValue at b: %d\n", b, c, *c);
```

You get the value of "b" (which is the address of "a"), the value of "c" (which is the address of "b:), and the dereferenced value from "c" (which is the address of "a") −

```
b: 6422036
Pointer to b is c: 6422024
Value at b: 6422036
```

Here, "c" is a double pointer. The first asterisk in its declaration points to "b" and the second asterisk in turn points to "a". So, we can use the double reference pointer to obtain the value of "a" from "c".

```
printf("Value of 'a' from 'c': %d", **c);
```

This should display the value of 'a' as 10.

Example

Here is the complete program that shows how a double pointer works –

```
</>                                                    Open Compiler
```

```c
#include <stdio.h>

int main(){

   int a = 10;
   int *b = &a;
   printf("a: %d \nAddress of 'a': %d \nValue at a: %d\n\n", a, b, *b);

   int **c = &b;
   printf("b: %d \nPointer to 'b' is c: %d \nValue at b: %d\n", b, c, *c);
   printf("Value of 'a' from 'c': %d", **c);

   return 0;
}
```

Output

Run the code and check its output –

```
a: 10
Address of 'a': 1603495332
Value at a: 10

b: 1603495332
Pointer to 'b' is c: 1603495336
Value at b: 1603495332
Value of 'a' from 'c': 10
```

A Double Pointer Behaves Just Like a Normal Pointer

A "pointer to pointer" or a "double pointer" in C behaves just like a normal pointer. So, the size of a double pointer variable is always equal to a normal pointer.

We can check it by applying the **sizeof operator** to the pointers "b" and "c" in the above program −

```
printf("Size of b - a normal pointer: %d\n", sizeof(b));
printf("Size of c - a double pointer: %d\n", sizeof(c));
```

This shows the equal size of both the pointers −

```
Size of b - a normal pointer: 8
Size of c - a double pointer: 8
```

Note: The size and address of different pointer variables shown in the above examples may vary, as it depends on factors such as CPU architecture and the operating system. However, they will show consistent results.

Multilevel Pointers in C (Is a Triple Pointer Possible?)

Theoretically, there is no limit to how many asterisks can appear in a pointer declaration.

If you do need to have a pointer to **"c"** (in the above example), it will be a "pointer to a pointer to a pointer" and may be declared as −

```
int ***d = &c;
```

Mostly, double pointers are used to refer to a two−dimensional array or an array of strings.